Experiment 8: Language Detection and N-gram Generation

Part A: Language Detection

```
# Lab Exercise 8(A): Language Detection

# Install required libraries
!pip -q install langdetect textblob langid

# Import libraries
from langdetect import detect, DetectorFactory
from textblob import TextBlob
import langid

# Make langdetect results more consistent
DetectorFactory.seed = 0

# Input text samples from the lab
texts = [
    "Bonjour tout le monde",
    "Hola amigos",
    "Hello everyone"
]

print("1. Language Detection using langdetect")
for text in texts:
    try:
        language = detect(text)
        print(f"{text} -> {language}")
    except Exception as e:
        print(f"Could not detect language for '{text}': {e}")

print("\n2. Language Detection using TextBlob")
for text in texts:
    try:
        blob = TextBlob(text)

        # Available only in some older TextBlob versions
        if hasattr(blob, "detect_language"):
            language = blob.detect_language()
            print(f"{text} -> {language}")
        else:
            print(
                f"{text} -> TextBlob language detection "
                "is unavailable in this version"
            )
    except Exception as e:
```

```
        print(f"{text} -> Detection failed: {e}")

print("\n3. Language Detection using langid")
for text in texts:
    try:
        language, score = langid.classify(text)
        print(f"{text} -> {language} (score: {score:.2f})")
    except Exception as e:
        print(f"Could not detect language for '{text}': {e}")
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 981.5/981.5 kB 18.5 MB/s eta 0:00
  Preparing metadata (setup.py) ... done
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.9/1.9 MB 56.9 MB/s eta 0:00:0
  Preparing metadata (setup.py) ... done
  Building wheel for langdetect (setup.py) ... done
  Building wheel for langid (setup.py) ... done
1. Language Detection using langdetect
Bonjour tout le monde -> fr
Hola amigos -> so
Hello everyone -> no

2. Language Detection using TextBlob
Bonjour tout le monde -> TextBlob language detection is unavailable in this v
Hola amigos -> TextBlob language detection is unavailable in this version
Hello everyone -> TextBlob language detection is unavailable in this version

3. Language Detection using langid
Bonjour tout le monde -> fr (score: -73.45)
Hola amigos -> es (score: 0.84)
Hello everyone -> it (score: -10.15)
```

Part B: N-gram Generation

```
# Lab Exercise 8(B): N-gram Generation

import nltk
from nltk.tokenize import word_tokenize
from nltk.util import ngrams

# Download tokenizer resources
nltk.download("punkt")
nltk.download("punkt_tab")

# Input sentence from the lab
text = "Natural language processing enables machines to understand te

# Tokenize the sentence
tokens = word_tokenize(text)
```

```
# Generate unigrams, bigrams, and trigrams
unigrams = list(ngrams(tokens, 1))
bigrams = list(ngrams(tokens, 2))
trigrams = list(ngrams(tokens, 3))

# Display results
print("Original Text:")
print(text)

print("\nTokens:")
print(tokens)

print("\nUnigrams:")
print(unigrams)

print("\nBigrams:")
print(bigrams)

print("\nTrigrams:")
print(trigrams)
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
Original Text:
Natural language processing enables machines to understand text

Tokens:
['Natural', 'language', 'processing', 'enables', 'machines', 'to', 'understar

Unigrams:
[('Natural',), ('language',), ('processing',), ('enables',), ('machines',), (

Bigrams:
[('Natural', 'language'), ('language', 'processing'), ('processing', 'enables

Trigrams:
[('Natural', 'language', 'processing'), ('language', 'processing', 'enables')
```